

Manual de rol — Persona A

Núcleo

Cresco · Versión 1.2 · 8 de agosto de 2026 · Envío del equipo: 28 de septiembre

Corrige la v1.0 del 3 de agosto de 2026. Ver [docs/99-archivo/ERRATA-2026-08.md](#) para el detalle de cada cambio. Esta versión 1.2 reinicia el calendario (tu semana 1 empieza el 9 de agosto, no el 4) y fija dónde vive tu `theme/Theme.ts` : dentro de `movil/` , que scaffoldea C el domingo.

Tu rol en una frase

Tú decides quién es quién y a qué tiene derecho.

Construyes la identidad, la estructura académica y —lo más importante— el sistema que garantiza que un representante solo vea a su propio hijo. Si eso falla, el producto muere. Ninguna otra parte del proyecto tiene ese peso.

Eres además quien desbloquea a los otros dos en la semana 1. Ellos pueden avanzar con el servidor simulado, pero necesitan tu base de datos y tu autenticación para trabajar con datos reales. Por eso tu semana 1 es la más intensa de las tres.

Parte 1 — Contexto del proyecto

1.1 Qué construimos

App Android de comunicación entre docentes y representantes legales de escuelas del Ecuador, para seguimiento de conducta. El estudiante nunca usa la app.

Dos interfaces: la del docente (herramienta de trabajo) y la del representante (informativa, simple).

1.2 El puntaje, en resumen

Cada estudiante parte de 60 puntos cada parcial. Piso 0, techo 100. Las acciones positivas suman (+1 o +2), las negativas restan (−1 a −3), con topes de +4 y −5 por día. Ese motor lo construye Persona B, pero tú necesitas conocerlo porque la matrícula que tú creas es la que lo sostiene.

1.3 Decisiones ya tomadas

#	Decisión
ADR-001	PostgreSQL + Drizzle ORM
ADR-002	Next.js como servidor, Expo para las apps
ADR-003	BetterAuth con plugin de organización
ADR-004	Permisos en dos capas: código + RLS
ADR-005	El puntaje es derivado, nunca un contador
ADR-006	RevenueCat es la fuente de verdad de pagos
ADR-007	Entrega sin tienda, repositorio abierto bajo AGPL-3.0

Parte 2 — Tu territorio

2.1 Tus tablas

Archivo: `db/schema/nucleo.ts`

`institucion · perfil_usuario · docente · representante · anio_lectivo · periodo_academico ·
dia_no_lectivo · curso · asignacion_docente · estudiante · matricula · invitacion_curso ·
vinculo_representacion · consentimiento`

2.2 Tus pantallas

App del docente

#	Pantalla	Semana
D1	Bienvenida y registro	2
D2	Inicio de sesión	2
D3	Crear curso (escuela, año lectivo, nivel)	2
D4	Definir parciales	2
D6	Panel del curso	2
D10	Invitar representantes	2
D7	Lista de estudiantes	3
D8	Aprobar estudiantes pendientes	3
D9	Importar CSV	4 (opcional)

App del representante

#	Pantalla	Semana
P2	Reclamar código del curso	3
P3	Datos del hijo y consentimiento	3
—	Selector de hijos (barra superior)	4

2.3 Tus endpoints

Todos están definidos en `api/openapi.yaml`. Si tu implementación difiere del contrato, el que está mal eres tú, no el contrato.

```
GET /docente/cursos <- D5 selector, D6 panel
POST /docente/cursos
POST /docente/cursos/{id}/periodos
POST /docente/cursos/{id}/invitacion
GET /docente/cursos/{id}/estudiantes
POST /docente/estudiantes/{id}/aprobar
POST /docente/cursos/{id}/estudiantes/importar
POST /representante/invitaciones/reclamar
GET /representante/hijos
```

2.4 Lo que NO tocas

- `db/schema/conducta.ts` — es de B
- `db/schema/interaccion.ts` — es de C
- Cualquier pantalla de reportes, acciones, citas o pagos

Si necesitas un campo en una tabla ajena, **lo pides en el grupo**. No lo agregas.

2.5 Superficie compartida

`db/schema/enums.ts` y `api/openapi.yaml` los tocan los tres. Cambiarlos exige avisar al grupo antes, no después.

Nota: el 8 de agosto se difirió el rol `DOCENTE_COLABORADOR` a la v2 — el campo sigue en el esquema (`ROL_ASIGNACION`) pero ninguna pantalla ni política RLS de la v1 debe construirse asumiendo que está activo.

2.6 Dónde vive tu código de aplicación

Tus pantallas (D1–D8, P1–P3) y tu archivo de dirección visual viven dentro de `movil/` — la única app Expo del proyecto, con enrutado por rol. No es una carpeta que tú creas: la scaffoldea C el domingo 9 de agosto, día 1 de la semana 1, junto con `servidor/`. Tú trabajas **dentro** de ella desde el lunes.

Eres además la dueña de `movil/theme/Theme.ts` y `movil/components/base/` — el resultado de la sesión de dirección visual (ver Parte 3bis). Cualquier pantalla, sea tuya, de B o de C, importa de ahí; nadie vuelve a escribir un `#hex` suelto.

Por qué no hay nada "compartido" entre `servidor/` y `movil/` más allá de `db/schema/` : el servidor calcula todo (puntaje, límites de plan) y lo devuelve ya resuelto en la respuesta JSON; el móvil solo llama a la API y renderiza. No hace falta `npm workspaces` ni un paquete compartido — cada carpeta tiene su propio `package.json` . Esto no reabre ADR-002 ("sin monorepo completo"). Ver decisiones S1-S6 en `Contexto/DECISIONS.md` .

Parte 3 — Preparación (semana 1, 4 de agosto en adelante)

Si nunca has usado estas herramientas, esta sección es la más importante del manual. Tómate el día completo. Llegar con todo instalado vale más que escribir código apurado el primer día.

3.1 Instalar

Node.js 20 o superior. Descárgalo de nodejs.org. Verifica:

```
node --version    # debe decir v20 o superior
npm --version
```

Git. Verifica con `git --version`. Configura tu identidad:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@correo.com"
```

Un editor. VS Code es lo más común. Instala las extensiones: ESLint, Prettier, y "PostgreSQL" para ver la base de datos.

Base de datos PostgreSQL. Tienes dos caminos:

- *Recomendado para empezar:* una base gratuita en la nube (Neon o Supabase). Te dan una `DATABASE_URL` lista y no instalas nada.
- *Local:* Docker con `docker run --name cresco-db -e POSTGRES_PASSWORD=clave -p 5432:5432 -d postgres:16`

3.2 Conceptos que necesitas entender antes de escribir código

¿Qué es un ORM? Una capa que te deja hablar con la base de datos usando TypeScript en vez de escribir SQL a mano. Drizzle es especial porque los tipos salen del esquema: si renombbras una columna, el compilador te avisa en todos los lugares que la usaban.

¿Qué es una migración? Un archivo SQL que lleva la base de datos de un estado al siguiente. Nosotros no las escribimos: las genera `drizzle-kit` leyendo el esquema. Nunca las edites a mano.

¿Qué es RLS? *Row Level Security*. Reglas dentro de PostgreSQL que filtran qué filas puede ver cada usuario, aunque la aplicación se equivoque. Es nuestra segunda línea de defensa.

¿Qué es BetterAuth? Una librería que resuelve registro, inicio de sesión y sesiones. Crea sus propias tablas (`user` , `session` , `account` , `organization` , `member`). Nuestra tabla `perfil_usuario` las **extiende** con cédula y teléfono; nunca duplica lo que ya está en `user` .

3.3 Clonar y levantar

```
git clone <url-del-repo>
cd cresco
npm ci                    # versiones exactas del lockfile
cp .env.example .env
# Abre .env y pega tu DATABASE_URL
npm run db:generate      # genera las migraciones desde el esquema
npm run db:migrate       # las aplica
```

Las correcciones E1 y E2 al esquema ya están aplicadas. No tienes que hacerlas: tienes que **no deshacerlas**. Se aplicaron y verificaron el 8 de agosto, antes del primer commit, según la decisión E3 de `DECISIONS.md` . Están explicadas en §4.1 y §4.3 para que sepas por qué el esquema se ve así.

Todo lo de arriba está verificado con Node 24.19.0: `npm ci` instala 36 paquetes sin errores, `npm run typecheck` pasa limpio, y la migración `0000_cresco_inicial.sql` produce 41 tablas con 57 CHECK.

Parte 4 — Semana a semana

Semana 1 · 9 al 15 de agosto — Cimientos

Esta es tu semana más pesada. B y C dependen de que termines. Si algo se atrasa, avísalo el mismo día en que lo notas, no el sábado.

C scaffoldea `servidor/` y `movil/` el domingo 9, antes de que tú empieces. No lo hagas tú ni por adelantado — es su tarea del día 1 (decisión S6).

Domingo 9 — Base de datos viva

1. Confirma que `npm run db:migrate` corre sin errores. El esquema ya trae E1 y E2 aplicadas (§4.1 y §4.3): no las rehagas.
2. Abre la base de datos con la extensión de VS Code y verifica que existen las 41 tablas.
3. Instala Drizzle Studio para inspeccionar datos visualmente:

```
npx drizzle-kit studio
```

Éxito: ves las tablas, puedes insertar una fila manualmente, y correr `npm run db:generate` responde *"No schema changes, nothing to migrate"*. Esa respuesta es tu comprobación permanente de que nadie desincronizó el esquema de la migración.

4.1 — Corrección: fechas que pueden capturar el día equivocado.

`matricula.fecha_ingreso` y `asignacion_docente.vigente_desde` son columnas `date` con `.defaultNow()`. En Postgres, `now()` convertido a `date` se resuelve según el timezone de la sesión de la base de datos — típicamente UTC en un proveedor cloud. Ecuador es UTC-5, así que una matrícula creada entre las 19:00 y las 23:59 hora de Guayaquil queda fechada al día siguiente, en silencio. La regla "todo en UTC, la conversión ocurre en la pantalla" (§5, más abajo) es correcta para `timestampz`, pero no cubre este caso de columnas `date`.

Ya está corregido (decisión E1, aplicada el 8 de agosto). Las dos columnas quedaron `NOT NULL sin default, a propósito`: obligan a que el servidor pase la fecha calculada con `REGLAS.ZONA_HORARIA (America/Guayaquil)`. En el SQL se ve así: `"fecha_ingreso" date NOT NULL`.

Si te topas con un error tipo *"null value in column fecha_ingreso violates not-null constraint"*, la respuesta correcta **no** es devolver el `.defaultNow()` — es pasar la fecha desde el servidor. Devolver el default reintroduce el bug en silencio.

Lunes 10 — Datos semilla

Crea `db/seeds/catalogo.ts` con:

- Las 6 categorías de acción (responsabilidad, disciplina, convivencia, puntualidad, desempeño, otro)
- Las 6 franjas de conducta con sus frases
- Los 5 planes
- Una plantilla de reporte con sus campos

Consejo: coordina con B. Las categorías y franjas son datos que él usa, pero la carga inicial la haces tú porque es parte del arranque del sistema. Acuerden quién escribe qué antes de duplicar trabajo.

Éxito: `npm run db:seed` deja la base lista y repetible. Ejecutarlo dos veces no duplica filas.

Martes 11 — BetterAuth

1. Instala y configura BetterAuth **dentro de** `servidor/`, el proyecto Next que C dejó armado el domingo.
2. Activa el plugin de organización.
3. Genera sus tablas con el CLI de BetterAuth.
4. Prueba registro e inicio de sesión desde el navegador.

Trampa común: BetterAuth genera una tabla `user`. No crees otra tabla de usuario. `perfil_usuario` apunta a `user.id` con una columna `user_id` de tipo texto. Si terminas con dos tablas de usuarios, tendrás que rehacerlo todo.

Éxito: te registras con correo y contraseña, y la fila aparece en la tabla `user`.

Miércoles 12 — La capa de acceso

Esta es la parte más importante de tu trabajo en todo el proyecto.

Crea la carpeta `db/acceso/` con funciones que **siempre** filtran por vínculo:

```
// db/acceso/estudiantes.ts
export async function estudiantesDelRepresentante(userId: string) {
  return db
    .select()
    .from(estudiante)
    .innerJoin(vinculoRepresentacion, eq(vinculoRepresentacion.estudianteId,
estudiante.id))
```

```

        .innerJoin(representante, eq(representante.id,
vinculoRepresentacion.representanteId))
        .where(and(
            eq(representante.userId, userId),
            eq(vinculoRepresentacion.estado, "ACTIVO"),
        ));
    }

```

Regla que anuncia al equipo: está prohibido usar `db.select()` suelto sobre `estudiante`, `matricula`, `accion_registrada` o `reporte_estudiante` fuera de `db/acceso/`. Tú rechazas cualquier PR que lo haga.

Éxito: existe al menos una función de acceso por cada tabla sensible.

Jueves 13 y viernes 14 — RLS

Escribe las políticas para las cinco tablas críticas. Patrón:

```

ALTER TABLE estudiante ENABLE ROW LEVEL SECURITY;

CREATE POLICY p_representante_lee_estudiante ON estudiante
FOR SELECT USING (
    EXISTS (
        SELECT 1 FROM vinculo_representacion v
        JOIN representante r ON r.id = v.representante_id
        WHERE v.estudiante_id = estudiante.id
            AND v.estado = 'ACTIVO'
            AND r.user_id = current_setting('app.user_id', true)
        )
    );

```

Cada petición debe abrir su transacción con:

```

SET LOCAL app.user_id = '<id del usuario>';

```

Consejo: si olvidas fijar la variable, la consulta devuelve **cero filas** en vez de todas. Eso es intencional: falla de forma visible y segura.

Éxito de la semana 1: un usuario se registra, inicia sesión, y una consulta directa sin `app.user_id` devuelve cero filas.

Semana 2 · 16 al 22 de agosto — El docente crea su curso

Pantallas D1, D2 (registro e inicio de sesión)

Usa los componentes de BetterAuth para Expo. No inventes tu propio manejo de sesión.

Pantallas D3, D4 (crear curso y parciales)

El flujo es un asistente de tres pasos:

1. Nombre de la escuela (texto libre) → crea `institucion` con `verificada = false`
2. Año lectivo: fecha de inicio y fin
3. Curso: nivel, paralelo, jornada

Luego, los parciales: entre 2 y 3, con fechas que no se solapan.

Trampa: si el docente ya alcanzó su límite de cursos (`DOC_FREE` permite 1), el endpoint debe devolver `402` con código `LIMITE_PLAN`. Coordina con C: él construye el paywall que se muestra al recibir ese código.

Pantalla D10 (invitación)

Genera un código corto legible (por ejemplo `5A-K7QM`) y un enlace. Vence a los 30 días. Admite varios usos porque invita al curso, no a un estudiante.

Éxito de la semana 2: un docente nuevo llega desde cero hasta tener un código de invitación en pantalla.

Semana 3 · 23 al 29 de agosto — Vinculación

Esta es la semana del hito crítico del proyecto.

Pantallas P2, P3 (canjear código y registrar al hijo)

El flujo del representante:

1. Ingresa el código corto
2. Registra los datos de su hijo: cédula, nombres, apellidos, fecha de nacimiento
3. Declara el parentesco
4. **Acepta el consentimiento de tratamiento de datos**, y se guarda la versión del documento aceptado

El estudiante queda con `estadoVerificacion = 'PENDIENTE'`.

Trampa 1 — segundo representante: si el estudiante ya tiene un representante activo, devuelve 409 . En la v1 hay un solo representante legal por estudiante. El índice único parcial ya lo impide; tu endpoint debe dar un mensaje claro en vez de dejar que explote un error de base de datos.

4.3 — Corrección: estudiantes sin documento pueden chocar entre sí.

`estudiante.numero_documento` es NOT NULL , y `TIPO_DOCUMENTO` admite `SIN_DOCUMENTO` — un caso real y esperado en niños pequeños. El índice único `ux_estudiante_documento` es sobre `(institucion, tipo_documento, numero_documento)` . Si dos estudiantes distintos del mismo colegio no tienen cédula, el valor que la app ponga en `numero_documento` para ambos puede colisionar — el segundo estudiante sin cédula recibiría un 409 como si fuera un duplicado, cuando es un niño distinto.

Ya está corregido (decisión E2, aplicada el 8 de agosto). `ux_estudiante_documento` es ahora un índice único **parcial**, el mismo patrón que ya usaban `ux_matricula_cursando` y `ux_curso_titular_vigente` . En el SQL se ve así:

```
CREATE UNIQUE INDEX "ux_estudiante_documento" ON "estudiante"
  USING btree ("institucion_id", "tipo_documento", "numero_documento")
  WHERE tipo_documento <> 'SIN_DOCUMENTO';
```

La unicidad sigue garantizada para quienes tienen cédula o pasaporte; los `SIN_DOCUMENTO` quedan fuera del índice y ya no colisionan entre sí. **Tu endpoint sigue debiendo dar un mensaje claro** en el caso legítimo de duplicado (mismo documento real ya registrado) en vez de dejar escapar el error de base de datos.

Pantallas D7, D8 (lista y aprobación)

El docente ve los estudiantes pendientes, puede corregir nombres y aprobar o rechazar. Al aprobar se crea la matrícula y el registro de puntaje inicial en 60.

Consejo: el docente va a aprobar 25 estudiantes de golpe. Haz que la pantalla permita aprobar varios a la vez, no uno por uno con tres toques cada uno.

Éxito de la semana 3: un representante registra a su hijo desde su teléfono y el docente lo aprueba desde el suyo.

Semana 4 · 30 de agosto al 5 de septiembre — Seguridad y multi-hijo

Selector de hijos

Barra superior con el nombre del estudiante actual y una flecha para cambiar. Funciona incluso con hijos en colegios distintos.

Las siete pruebas de seguridad

Son obligatorias. Están en `docs/01-arquitectura/matriz-permisos.md` :

#	Caso	Esperado
S-1	Representante A pide el reporte del hijo de B	403
S-2	Vínculo revocado	403
S-3	Docente registra acción en curso ajeno	403
S-4	Matrícula finalizada	403
S-5	Consulta sin app.user_id	0 filas
S-6	Segundo representante para el mismo estudiante	409
S-7	Alerta sin reautenticación	401

Importación CSV (opcional)

Si hay tiempo. Se parsea el archivo y **se descarta**: nunca se almacena.

Éxito de la semana 4: las siete pruebas pasan en el CI.

Semanas 5 y 6 · 6 al 19 de septiembre

- Guías de uso para docente y representante (dos páginas con capturas)
- Ajustes del flujo de vinculación según lo que hayan visto probando
- Semana 6: trabajo conjunto en datos de demostración, video y cierre

Parte 5 — Trampas y consejos

No dupliques la identidad. BetterAuth ya tiene `user`. Tu `perfil_usuario` la extiende. Es el error más caro de deshacer.

La invitación de BetterAuth no es tu invitación. BetterAuth tiene una invitación para sumar adultos a una organización. La tuya (`invitacion_curso`) es del dominio y desemboca en registrar un menor con consentimiento. No las mezcles.

Nada se borra. Revocar un vínculo corta el acceso hacia adelante; no borra el histórico. Al finalizar una matrícula, el representante pierde acceso pero la institución conserva los datos.

Todo en UTC — con un matiz. Los `timestampz` se guardan en UTC; la conversión a `America/Guayaquil` ocurre en la pantalla, nunca en la base de datos. Pero las columnas `date` que representan "qué día ocurrió esto" (como `fecha_ingreso`) las calcula el servidor explícitamente con la zona horaria correcta — nunca con `defaultNow()` . Ver §4.1.

Prueba con dos cuentas desde el primer día. Crea dos representantes y dos estudiantes en tus datos de prueba — uno de ellos sin documento. Es la única forma de notar una fuga de permisos o una colisión de constraint antes de que sea tarde.

La cédula ecuatoriana se valida localmente. El algoritmo del dígito verificador (módulo 10) son unas veinte líneas. No busques una API del Registro Civil: no está abierta al público y consultar identidades reales complica el cumplimiento de protección de datos.

No construyas nada para `DOCENTE_COLABORADOR` . Está diferido a v2 desde el 8 de agosto.

Parte 6 — Cuando te bloques

1. **Relee el contrato.** ¿ `openapi.yaml` ya define el comportamiento?
2. **Revisa el ADR.** ¿La decisión ya está tomada?
3. **Medio día máximo.** Si llevas más, avisa al grupo ese mismo día.
4. **Pregunta a una IA** pegando el bloque de contexto de la última sección.

Parte 7 — Bloque de contexto para IA

Copia todo esto y pégalo antes de tu pregunta.

CONTEXTO – PROYECTO CRESCO, ROL: PERSONA A (NÚCLEO)

Soy uno de 3 estudiantes (equipo Neofix) construyendo una app Android para el hackathon RevenueCat Shipaton 2026, categoría Next Gen. Semana 1 desde el 9 de agosto de 2026; deadline interno 20 de septiembre; envío real 28 de septiembre. Es mi primera app. Mi rol es el módulo de identidad, estructura académica, vinculación y permisos.

MI CÓDIGO VIVE EN `movil/theme/Theme.ts`, `movil/components/base/` y dentro de `movil/` (pantallas D1-D8, P1-P3) y `servidor/` (BetterAuth). Esas dos carpetas las scaffoldea C el día 1 con `create-next-app / create-expo-app`; yo trabajo dentro, no las creo.

EL PRODUCTO: app de comunicación entre docentes y representantes legales de escuelas del Ecuador, para seguimiento de conducta. El estudiante NUNCA es usuario. Dos interfaces: docente y representante.

STACK: PostgreSQL + Drizzle ORM. Next.js como servidor (BetterAuth, rutas API). Expo/React Native para las apps. BetterAuth con plugin de organización. NativeWind para estilos (shadcn NO funciona en React Native).

MIS TABLAS: `institucion`, `perfil_usuario`, `docente`, `representante`, `anio_lectivo`, `periodo_academico`, `dia_no_lectivo`, `curso`, `asignacion_docente`, `estudiante`, `matricula`, `invitacion_curso`, `vinculo_representacion`, `consentimiento`.

REGLAS QUE ME TOCAN:

- No hay administrador de institución en v1. El docente escribe el nombre de su escuela como texto libre; la institución queda con `verificada=false`.
- No hay rol `DOCENTE_COLABORADOR` activo en v1 (diferido a v2); el campo existe en el esquema pero ninguna pantalla ni política lo usa.
- La organization de BetterAuth ES la institución. `perfil_usuario` EXTIENDE el user de BetterAuth (cédula, teléfono); no lo duplica.
- Una persona puede ser docente y representante a la vez: son dos tablas de perfil separadas, no un campo "rol".
- La invitación es al CURSO (no a un estudiante), dura 30 días, admite

varios usos.

- El REPRESENTANTE registra los datos de su hijo al canjear el código; queda en estado PENDIENTE hasta que el docente lo apruebe. Al aprobar nace la matrícula con puntaje base 60.
- UN SOLO representante legal por estudiante en la v1 (índice único parcial). Intentar un segundo devuelve 409.
- El consentimiento de tratamiento de datos del menor se guarda con la versión del documento aceptado, no como booleano.
- Un estudiante tiene una sola matrícula CURSANDO. Estados: CURSANDO, RETIRADA, TRASLADADA, FINALIZADA.
- El histórico de conducta cuelga de la matrícula, no del estudiante.
- Nada se borra físicamente: se anula o se cambia de estado.
- Timestamps timestamptz en UTC; zona America/Guayaquil. Las columnas `date` que representan un día calendario (fecha_ingreso, vigente_desde) las calcula el servidor explícitamente con esa zona horaria – nunca defaultNow(), porque now()::date puede resolver al día siguiente según el timezone de la sesión de la base de datos.
- estudiante.numero_documento es NOT NULL, pero el índice único ux_estudiante_documento debe ser PARCIAL (excluir tipo_documento = 'SIN_DOCUMENTO'), para que dos estudiantes sin cédula del mismo colegio no colisionen entre sí.

SEGURIDAD (lo más crítico de mi rol): un representante solo accede a estudiantes con vínculo ACTIVO. DOS capas:

1. Capa obligatoria db/acceso/ en el servidor: prohibido usar db.select() suelto sobre estudiante, matricula, accion_registrada, reporte_estudiante.
2. Políticas RLS de PostgreSQL sobre esas mismas tablas. Cada petición abre transacción con SET LOCAL app.user_id = '<id>'. Si se olvida, devuelve 0 filas.

CONVENCIONES: tablas y columnas en español snake_case. Estados en MAYUSCULA_SNAKE definidos en db/schema/enums.ts. Claves primarias uuid. Migraciones generadas por drizzle-kit, nunca escritas a mano.

MI RESTRICCIÓN: 6 semanas, primera app, prefiero soluciones simples y probadas sobre elegantes. Explícame los conceptos, no solo el código.